

BiScale: Energy-Efficient Disaggregated LLM Serving via Phase-Aware Placement and DVFS

Omar Basit*, Yunzhao Liu*, Z. Jonny Kong, Y. Charlie Hu
Purdue University

Abstract

Prefill/decode disaggregation is increasingly adopted in LLM serving to improve the latency-throughput tradeoff and meet strict TTFT and TPOT SLOs. However, LLM inference remains energy-hungry: autoscaling alone is too coarse-grained to track fast workload fluctuations, and applying fine-grained DVFS under disaggregation is complicated by phase-asymmetric dynamics and coupling between provisioning and frequency control.

We present BiScale, a two-tier energy optimization framework for disaggregated LLM serving. BiScale jointly optimizes placement and DVFS across prefill and decode using predictive latency and power models. At coarse timescales, BiScale computes phase-aware placement and baseline frequencies that minimize energy while satisfying SLO constraints. At fine timescales, BiScale dynamically adapts GPU frequency per iteration using stage-specific control: model predictive control (MPC) for prefill to account for queue evolution and future TTFT impact, and lightweight slack-aware adaptation for decode to exploit its smoother, memory-bound dynamics. This hierarchical design enables coordinated control across timescales while preserving strict serving SLOs.

Evaluation on a 16xH100 cluster serving Llama 3.3 70B with production-style traces shows that BiScale meets TTFT/TPOT SLOs while reducing energy by up to 39% in prefill and 48% in decode relative to DistServe.

1 Introduction

Recent frontier models such as GPT-5, Claude, Kimi K2, and DeepSeek-V3 [5, 17, 23, 32] now power a wide range of user-facing services, including chatbots, coding assistants, and enterprise agents. However, these models impose substantial compute and memory demands per request, making it fundamentally challenging to simultaneously meet strict latency SLOs and sustain high throughput. In these interactive settings, user experience depends on strict latency service-level objectives. In particular, users expect low *Time To First Token* (TTFT) and stable *Time Per Output Token* (TPOT). At the same time, providers must maintain high throughput to keep serving costs under control.

To address this tension between strict latency SLO targets and high throughput, recent LLM serving systems increasingly adopt prefill/decode disaggregation, where prompt processing and autoregressive generation are provisioned separately. Recent systems such as DistServe [46] and

Splitwise [34] show that this design improves the latency-throughput tradeoff and helps meet strict TTFT and TPOT SLOs. The same direction is visible in industry: DeepSeek-V3 [23] and serving stacks such as NVIDIA Dynamo and vLLM [31, 40] all support disaggregated serving.

Despite these efficiency gains, the energy footprint of LLM inference remains substantial. Recent production-oriented analysis estimates a median of 0.34 Wh per query for frontier-scale models on H100-class inference systems, and up to 4.32 Wh per query under test-time scaling workloads [27]. At deployment scale, this corresponds to roughly 0.8–1.8 GWh/day for a system serving 1B queries per day [27]. At the serving-system level, recent characterization work reports that inference clusters often operate with significant power overhead under time-varying workloads [30, 33].

Many serving systems reduce energy indirectly via autoscaling, by adapting active resources to time-varying workload demand [10, 11, 44, 45]. However, these mechanisms operate at coarse timescales because reconfiguration incurs non-trivial overheads, such as model weight loading, and therefore cannot swiftly follow fine-grained workload fluctuations [16]. Therefore, they need to over-provision according to the peak load over the period of time of each configuration, leading to inefficiencies during non-peak times.

Modern GPUs support Dynamic Voltage and Frequency Scaling (DVFS), which allows the system to adjust SM clock frequency (and corresponding voltage states) at fine time scales, e.g., on the order of tens of milliseconds. DVFS has been successfully applied in LLM training to reduce energy consumption by up to 30% with negligible performance impact [8, 42]. However, applying DVFS to online inference presents fundamentally new challenges. First, prefill and decode have distinct bottlenecks and latency sensitivities, and must continuously satisfy their respective SLOs (TTFT for prefill and TPOT for decode) as the workload fluctuates over time; second, under disaggregation, energy efficiency depends jointly on cluster placement decisions and DVFS settings across both phases.

Several recent LLM serving systems have exploited DVFS to reduce the energy drain, including DynamoLLM [39] and throttLL'eM [14]. However, these work focus on non-disaggregated architectures, and therefore do not address the joint optimization required in disaggregated serving.

Motivated by this gap, in this paper, we study minimizing energy consumption in prefill/decode-disaggregated LLM serving. Unlike prior non-disaggregated energy-efficient

* These authors contributed equally to this work.

serving settings, disaggregation introduces additional coupling across stages and timescales, leading to several new challenges:

- *How to pick cluster placement configurations under a large search space?* In disaggregated serving, provisioning must jointly configure *both* prefill and decode pools. A placement must choose, for each phase, the number of instances, parallelism configurations, baseline frequencies, and routing weights. The cross-product of these phase-specific choices yields a large combinatorial search space. Moreover, the two phases are tightly coupled: under-provisioning prefill degrades TTFT, while under-provisioning decode creates post-prefill backlogs and inflates TPOT, even if the other phase is over-provisioned. Because input/output length distributions evolve over time and shift load pressure between phases over minutes, the energy-optimal placement must be recomputed efficiently to track workload changes.
- *How should DVFS control adapt to phase-asymmetric dynamics?* Prefill and decode have fundamentally different bottlenecks, memory pressure, and temporal behavior, which makes uniform DVFS ineffective. Prefill is typically compute-bound and highly frequency-sensitive, whereas decode is often memory-bandwidth-bound and retains large KV caches across many iterations, creating sustained memory pressure. Prefill load is arrival-driven and bursty, while decode load evolves more smoothly as requests remain in decode for many iterations. Effective DVFS therefore must be phase-aware and tailored to each stage’s dynamics to preserve TTFT/TPOT SLOs.

To address these challenges, we design BiScale, a two-tier energy optimization framework for disaggregated LLM serving. BiScale is built on the key insight that energy-efficient serving requires coordinated control across both coarse-grained provisioning and fine-grained DVFS, while respecting the distinct dynamics of prefill and decode.

At Tier 1, BiScale computes phase-aware placement and baseline frequency configurations at coarse timescales using predictive latency and power models. Tier 1 identifies an energy-minimizing operating point that satisfies TTFT and TPOT SLOs under expected peak load, ensuring system feasibility.

At Tier 2, BiScale dynamically adjusts GPU frequency at iteration granularity to exploit short-term workload slack and correct prediction errors. BiScale employs stage-specific control strategies tailored to phase dynamics. For prefill, where frequency decisions affect queue evolution and future TTFT, BiScale uses model predictive control (MPC) to minimize energy while preserving latency headroom over a multi-batch horizon. For decode, which exhibits smoother and predominantly memory-bound behavior, BiScale applies lightweight per-iteration frequency adaptation to safely harvest slack.

Both tiers rely on data-driven latency and power models trained offline, enabling accurate prediction of performance

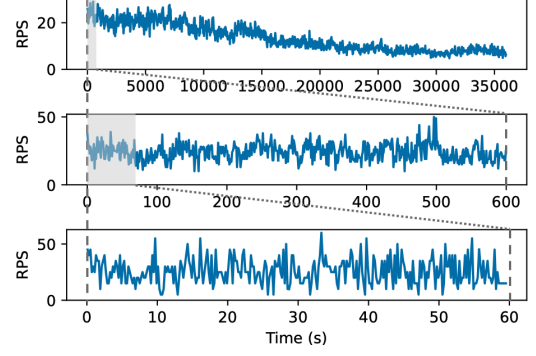


Figure 1. The RPS timelines of the Azure LLM inference trace [6] over 10 hours, 10 minutes, and 1 minute.

and energy across configurations. Together, this hierarchical, model-driven design enables BiScale to jointly optimize placement and DVFS across timescales, substantially reducing energy while preserving strict serving SLOs.

We implement BiScale and evaluate it using Llama 3.3 70B on a 16×H100 cluster. We implement BiScale and evaluate it using Llama 3.3 70B on a 16×H100 cluster. Both tiers contribute to the overall energy savings. Relative to DistServe, Tier 1 alone (PlaceOnly) reduces prefill energy by up to 29% and decode energy by up to 45%. Tier 2 (BiScale) further improves energy efficiency via fine-grained DVFS, increasing the reduction to up to 39% for prefill and 48% for decode.

In summary, this paper makes the following contributions:

- We present, to our knowledge, the first energy-efficient LLM inference system under prefill/decode disaggregation. We characterize how phase asymmetry and workload dynamics jointly shape the energy-SLO tradeoff.
- We design BiScale, a two-tier solution that combines phase-aware coarse-grained placement with fine-grained online DVFS. The design explicitly captures placement–DVFS coupling and uses DVFS as an online correction mechanism for workload-prediction error.
- We implement and evaluate BiScale on production-style traces and a real multi-GPU testbed. Results show substantial energy reductions over strong baselines while preserving strict TTFT/TPOT SLOs.

2 Background

2.1 LLM Online Serving Workload Dynamics

Online user-facing LLM inference systems receive requests with workload that dynamically changes over time. In Figure 1, we show the RPS timeline from the Azure LLM inference trace [6] across multiple time scales. The trace shows that the workload exhibits burstiness consistently across different timescales – from hours to minutes to seconds.

To quantify this workload fluctuation, we show the *normalized variance-time plot* [12, 19] of the full trace, which spans 7 days. To calculate normalized variance-time, we first divide the trace into non-overlapping windows with sizes

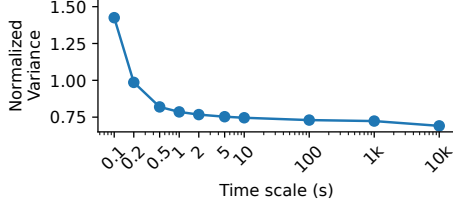


Figure 2. Variance-time plot of request-per-second (RPS) in the Azure LLM inference trace [6]. The trace exhibits notable fluctuation across both short and long timescales, with slightly greater variance observed at shorter timescales.

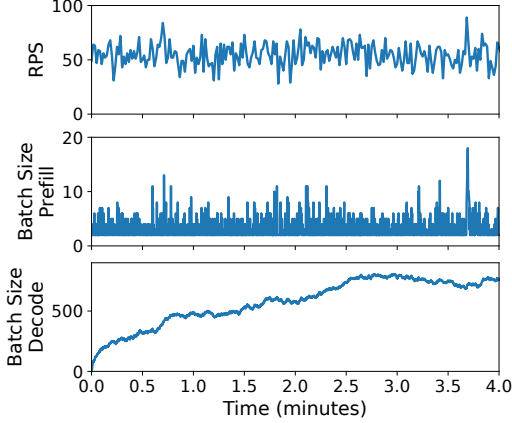


Figure 3. Number of running requests in prefill and decode instances plotted with workload in RPS.

ranging from 0.1 to 10000 seconds. For each window, we compute the request per second (RPS), then calculate the mean and variance of these RPS values across all windows. We plot the normalized variance, defined as the ratio of variance to mean, as a function of window size, as shown in Figure 2.

We observe significant workload variability across multiple time scales. At the hourly level (1,000–10,000s), the normalized variance stabilizes around 0.7, reflecting diurnal and daily load cycles. At the minute scale (100–1,000 s), the variance increases further. At the second scale (down to 0.1 s), the trace exhibits sharp RPS swings, with normalized variance peaking at 1.4. This shows that online LLM serving workloads vary significantly at both long and short timescales, and LLM inference systems must be designed to tolerate this variability.

2.2 Energy Efficient LLM Serving

To reduce the energy consumption of LLM inference serving, existing systems typically rely on two complementary mechanisms that operate at different time scales. First, many existing systems employ autoscaling to address long-term workload variations [9, 28, 29, 36, 44, 45]. By dynamically adjusting the number of active serving instances based on historic request rates, autoscaling reduces idle resources during low-load periods and hence improves overall utilization.

However, autoscaling is limited by high reconfiguration overheads in LLM serving, including model loading and request draining, which typically take several minutes [16]. As a result, it cannot respond to fine-grained workload fluctuations.

Second, Dynamic Voltage and Frequency Scaling (DVFS) reduces energy consumption by adjusting GPU operating frequency. Unlike autoscaling, DVFS can operate at both long and short time scales, ranging from coarse-grained power provisioning to fine-grained adaptation under bursty workloads. Prior works have shown that GPUs often run at unnecessarily high frequencies during memory-bound or lightly loaded phases, wasting energy [14, 38, 39]. Accordingly, these works reduce power consumption by lowering GPU operating frequency when possible.

Recent systems integrate these ideas with inference-specific optimizations. For example, DynamoLLM [39] improves energy efficiency by partitioning GPUs into pools based on input and output lengths, and selects the optimal parallelism and frequency configurations within each pool. throttLL’eM [14] applies predictive DVFS to throttle GPU frequency during inference execution, reducing energy while maintaining latency SLOs.

2.3 Prefill/Decode Disaggregation

In many existing LLM serving systems [4, 18, 36, 43], the prefill and decode phases are collocated on the same GPU, interleaving the compute-bound prefill of new requests with the memory-bound decode of ongoing requests, which is known as continuous batching [43]. However, such systems suffer from significant inter-phase interference. Prefill workloads are bursty and compute-intensive; when a large prefill request is scheduled, it can delay the execution of ongoing decode steps, leading to spikes in inter-token latency. This effect, effectively a form of head-of-line blocking, makes it difficult to simultaneously satisfy the service level objectives (SLOs) for prefill and decode, namely time-to-first-token (TTFT) and time-per-output-token (TPOT). Prior work has proposed mitigations such as chunked prefill [4], which limits the number of tokens processed per iteration by splitting prefills into smaller chunks. However, selecting an appropriate chunk size that balances TTFT and TPOT remains challenging and is workload-dependent.

To address this problem, recent systems adopt **Prefill-Decode (P/D) Disaggregation** [34, 46], which partitions the GPUs into two specialized pools of instances: 1) Prefill instances, which process prompts and compute the initial key-value (KV) cache. 2) Decode instances, which are dedicated to autoregressive token generation. Once a request finishes prefill on a prefill instance, it is then forwarded to a decode instance along with its KV cache. In this disaggregated setup, prefill and decode no longer interfere with each other, and the two phases can both maintain low and stable latencies. This separation also enables independent scaling of the two stages, allowing resources to be allocated

according to the prefill-to-decode workload ratio. For example, DistServe reports up to a 7.4× improvement in goodput over colocated baselines, while meeting TTFT and TPOT SLOs.

Beyond improved goodput, disaggregated architectures provide greater flexibility for system reconfiguration under the presence of fluctuating workloads. Although dynamic autoscaling is commonly used in online services to improve utilization [25], its effectiveness in LLM serving is limited by high reconfiguration costs. Reconfiguring a serving instance requires loading large model parameters into GPU memory and draining in-flight requests, incurring substantial latency [11]. As a result, reconfiguration is typically coarse-grained. Disaggregation mitigates this limitation by allowing prefill and decode pools to scale independently, enabling targeted adjustments to phase-specific demand and reducing the frequency and impact of disruptive reconfigurations compared to colocated systems.

2.4 The Energy Efficient P/D Disaggregation Serving Problem

Given the benefits of P/D disaggregation and the growing importance of energy efficiency, in this paper we study the following problem: *How to minimize the energy consumption of a two-level P/D-disaggregated LLM inference serving system while meeting strict TTFT and TPOT SLOs?*

3 Challenges

By separating the prefill and decode phases onto different execution pools, disaggregation improves SLO attainment but introduces new challenges for energy-efficient serving. In particular, disaggregation exposes pronounced *asymmetry* in workload characteristics, resource requirements, and SLO sensitivity between the two phases. This asymmetry complicates both resource provisioning (to exploit coarse-grained traffic variability) and DVFS control (to exploit-grained traffic variability).

3.1 Key Observation: Phase-specific Workload Characteristics

A key observation in LLM serving is that the prefill and decode phases exhibit fundamentally different workload characteristics [4]. First, since prefill phase is typically compute-bound whereas decode phase tends to be memory-bandwidth-bound, prefill and decode latencies have different sensitivities to input length and GPU clock frequency. For example, prior work [38] shows that prefill latency (i.e., TTFT) is more heavily affected by input length and GPU frequency, while decode latency (i.e., TPOT) is less sensitive to frequency and is largely insensitive to input length on modern GPUs that have high memory bandwidth.

The distribution of request input and output lengths further affects resource demand and consequently how DVFS

should be applied across prefill and decode. Workloads with long prompts and short responses place more pressure on prefill, whereas workloads with short prompts and long generations stress decode throughput and memory bandwidth.

3.2 Key Challenges

The above phase-specific bottlenecks and workload variations present a unique set of challenges in exploiting resource provisioning and DVFS control to optimize energy efficiency in a P/D disaggregated serving system.

C1: Combinatorial search space of energy-efficient resource provisioning. At coarse timescales, resource provisioning periodically reconfigures the cluster, i.e., the placement of GPUs across serving instances, to minimize cluster energy while meeting TTFT/TPOT SLOs for the next workload window. In non-disaggregated systems, provisioning only needs to choose the number of instances and their configurations (e.g., tensor-parallel degree and operating frequency). Under P/D disaggregation, however, the system must provision *both* prefill and decode pools, and the two decisions are tightly coupled: under-provisioning prefill degrades TTFT, while under-provisioning decode creates post-prefill backlogs and inflates TPOT; over-provisioning either phase wastes energy or simply shifts the bottleneck to the other phase. Consequently, provisioning must jointly choose phase-specific instance counts and configurations, yielding a large combinatorial configuration space.

This problem is further complicated by workload variability. Because prefill and decode have different resource bottlenecks and the input/output length distribution evolves over time, the dominant load can shift between phases, changing the energy-optimal provisioning. As we show in §6.2.2, these shifts can occur within minutes. Therefore, *provisioning must search a large space and do so fast enough to continuously track workload shifts.*

C2: Phase-specific workload characteristics complicates DVFS control. Exploiting fine-grained workload variability using DVFS is particularly challenging under P/D disaggregation due to fundamental differences between prefill and decode. First, as discussed in §3.1, the two phases have distinct performance bottlenecks and frequency sensitivities: prefill is typically compute-bound and highly sensitive to GPU frequency, whereas decode is often memory-bandwidth-bound and less responsive to frequency scaling. Second, the two phases differ significantly in GPU memory pressure. While prefill instances process prompts without retaining large KV caches, decode instances must maintain per-request KV states across many iterations, creating sustained GPU memory pressure and increasing the risk of out-of-memory (OOM) conditions. DVFS decisions must therefore account for both compute performance and memory feasibility.

Third, prefill and decode exhibit distinct temporal dynamics. Decode load evolves smoothly because each request remains in the decode stage for many iterations—one per generated token—often up to thousands. In contrast, prefill load is arrival-driven and can appear as short-lived but intense spikes, since a request typically completes prefill in a single iteration (or a small number of iterations with chunked prefill [4]). Figure 3 illustrates this effect by plotting prefill and decode batch sizes as proxies for phase-specific load intensity. We see that prefill (middle plot) exhibits substantially higher variability than decode (bottom plot).

These differences in bottlenecks, memory pressure, and temporal dynamics make it difficult to apply a uniform DVFS policy across both phases. Instead, *effective DVFS must be phase-aware and responsive to the distinct characteristics of prefill and decode.*

4 Design

4.1 Design Principles

Our objective is to minimize the energy consumption of a P/D-disaggregated LLM serving system while satisfying strict TTFT and TPOT SLOs under dynamically varying workloads. As discussed in §3, this problem is challenging due to (i) the asymmetric performance characteristics of prefill and decode, and (ii) time-varying workload intensity and input/output length distributions.

Key insight: hierarchical control with coordinated feasibility and adaptation. BiScale is built on the observation that energy-efficient serving requires separating feasibility enforcement from fine-grained energy optimization, while coordinating both through predictive models. Placement decisions determine the feasible operating region by provisioning sufficient compute, memory, and latency headroom. Within this feasible region, fine-grained DVFS can safely exploit transient workload slack to reduce energy.

To realize this insight, BiScale adopts a *two-tier control architecture* operating at distinct timescales.

Tier 1: Phase-aware coarse-grained provisioning. Every provisioning window (e.g., 5 minutes), Tier 1 determines (i) the number of prefill and decode instances, (ii) the tensor parallel (TP) degree of each instance, (iii) a baseline operating frequency for each instance, and (iv) request routing weights. Tier 1 uses predictive latency and power models to identify an energy-minimizing configuration that satisfies TTFT and TPOT constraints under expected peak load. This ensures that all subsequent DVFS actions operate within a provably feasible region.

Tier 2: Phase-specific fine-grained DVFS control. At iteration granularity, Tier 2 dynamically adjusts GPU frequency to exploit short-term workload slack while preserving SLOs. Tier 2 never violates feasibility guarantees established by Tier 1. Instead, it adapts frequency to workload dynamics, correcting prediction errors and harvesting slack

caused by burstiness and load variation. Tier 2 applies stage-specific control strategies that account for the distinct temporal and resource characteristics of prefill and decode.

This hierarchical decomposition separates long-term capacity provisioning from short-term energy adaptation, enabling scalable, model-driven optimization across both timescales while preserving strict SLO guarantees.

4.2 System Architecture

Figure 4 illustrates the BiScale architecture, which consists of an offline profiling phase and an online serving phase. The online phase includes a *data plane* (prefill and decode instances that execute inference) and a *control plane* (a coarse-grained provisioning controller and a fine-grained DVFS controller).

Offline profiling. BiScale trains latency and power models for each (LLM model, GPU type) pair by profiling execution across different batch shapes, tensor-parallel (TP) degrees, and GPU frequencies. These models capture iteration-level performance and energy behavior and are used by both controllers at runtime.

Online serving: request path and data plane. When a request arrives at the frontend, it is first sent to the router (①). The router selects a prefill instance based on the current routing weights and forwards the request (②). Inside the prefill instance, the request enters a waiting queue (③) and is later scheduled for execution (④). After prefill completes, the request returns to the router (⑤), which forwards it to a decode instance according to the routing policy. The request then queues at the decode instance (⑥) and is scheduled for token generation (⑦); during decode, the instance retrieves the corresponding KV cache produced by prefill.

Online serving: control plane. The coarse-grained provisioning controller periodically analyzes recent workload history (recorded at request arrival in ①) and computes a placement plan for the next provisioning window, including instance counts, TP degrees, routing weights, and baseline frequencies. It launches or tears down prefill and decode instances as needed and updates the router’s load-balancing weights.

The fine-grained DVFS controller continuously monitors runtime statistics from each instance, including queue lengths and batch characteristics (collected during ③–④ for prefill and ⑥–⑦ for decode), and adjusts GPU frequency at iteration granularity via a lightweight frequency modulator. The DVFS controller targets the TTFT SLO for prefill and the TPOT SLO for decode while minimizing power consumption.

Both controllers rely on the offline-trained latency and power models to predict SLO feasibility and energy under candidate configurations, enabling coordinated, energy-efficient placement and DVFS decisions under dynamically varying workloads.

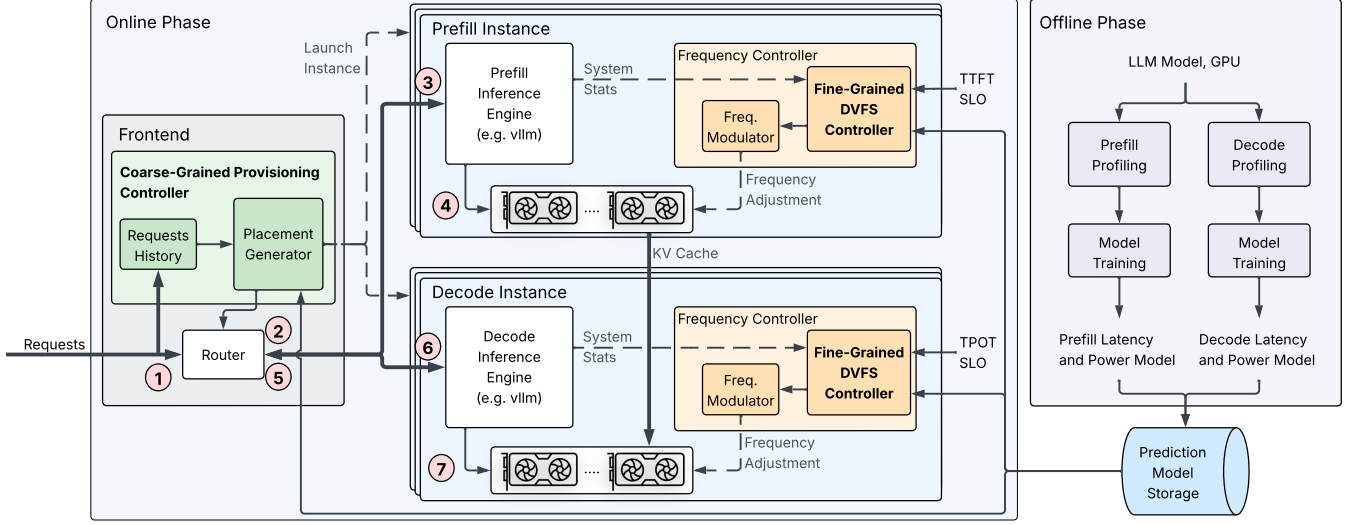


Figure 4. Architecture overview of BiScale.

4.3 Tier 1: Coarse-Grained Provisioning

Tier 1 establishes an energy-minimizing operating point that guarantees SLO feasibility under predicted peak load, defining a safe operating region within which Tier 2 can dynamically adapt frequency.

4.3.1 The Cluster Provisioning Problem. Tier 1 reacts to longer-term workload shifts. by periodically, e.g., every 5 minutes, determining a placement configuration for prefill and decode that minimizes the total energy of the cluster while meeting TTFT/TPOT SLOs at a target goodput R , i.e., the predicted peak request rate during the next provision window. Similar to prior work such as DistServe [46], BiScale predicts the next-window workload, e.g., request trace, using recent request history.

Specifically, we define a *serving instance configuration* as a tuple:

$$(type, TP\ degree, frequency)$$

where $type \in \{prefill, decode\}$. A *placement configuration* for the cluster then specifies (1) the number of instances of each serving instance configuration, (2) the TP degree of each instance, (3) the baseline GPU frequency of each instance, and (4) routing weights across instances.

In addition to SLO feasibility, Tier 1 must ensure memory feasibility for both phases under the predicted input/output length distribution.

4.3.2 Placement Optimization via ILP. Provisioning prefill and decode jointly creates a large combinatorial search space due to the cross-product of instance types, TP degrees, and frequencies. To efficiently find an optimal configuration under resource and SLO constraints, BiScale formulates provisioning as an integer linear programming (ILP) problem.

Given a total target goodput R , the ILP outputs the optimal placement that minimizing energy under cluster capacity and SLO constraints, which includes:

- how many prefill and decode instances to launch;
- the TP degree and baseline frequency of each instance;
- and the routing weights across instances.

To this end, the ILP takes as input a *configuration table* that maps each candidate prefill or decode instance configuration c to three values: (R_c, E_c, G_c) , where R_c is maximum SLO-feasible goodput, E_c is energy per request, and G_c is GPU cost. We describe how this table is constructed in §4.3.3. Let n_c denote the number of instances of configuration c , G be total GPUs available, and α be a safety margin to compensate for modeling and subsampling inaccuracies. The provisioning problem is formulated as:

$$\text{minimize } \sum_c n_c E_c R_c \quad (1)$$

$$\text{s.t. } \sum_c n_c G_c \leq G \quad (2)$$

$$\sum_{c \in \text{prefill}} n_c R_c \geq (1 + \alpha) R \quad (3)$$

$$\sum_{c \in \text{decode}} n_c R_c \geq (1 + \alpha) R \quad (4)$$

$$n_c \in \mathbb{N} \quad (5)$$

The first constraint ensures the total GPU usage does not exceed cluster capacity. The second and third constraints ensure that prefill and decode independently have sufficient capacity to handle the target goodput with margin α . This separation is critical in disaggregated serving, since SLO violations in either phase cannot be compensated by over-provisioning the other phase.

The ILP solution determines the number and configuration of prefill and decode instances, along with their baseline

operating frequencies. Routing weights are then derived proportionally based on instance capacity.

4.3.3 Configuration Table Construction. A key input to the ILP is the configuration table, which maps each candidate instance configuration c to a tuple (G_c, R_c, E_c) representing GPU cost, maximum SLO-feasible goodput, and energy per request for a given input trace. Constructing this table is challenging because these quantities depend on workload-dependent batching, queueing, and frequency-sensitive execution dynamics, which are difficult to capture using closed-form analytical models.

To address this, BiScale uses a frequency- and memory-aware, iteration-level inference simulator. The simulator is built on top of offline-trained latency and power models (§4.5) and accurately reproduces batched inference execution under realistic workload traces. The simulator takes as input (1) a request trace, (2) the instance type (prefill or decode), (3) tensor-parallel (TP) degree, and (4) GPU frequency. Given these inputs, it simulates per-iteration batching, scheduling, and execution while tracking request-level TTFT and TPOT, iteration-level power consumption, and KV cache usage. This allows BiScale to determine both SLO feasibility and memory feasibility for each configuration under the given workload.

Using this simulator, BiScale computes the configuration table entries as follows.

GPU cost G_c . For a configuration c with tensor-parallel degree TP_c , each shard occupies one GPU. Thus, the total GPU cost of one instance is: $G_c = TP_c$. This formulation naturally extends to other parallelism schemes (e.g., pipeline parallelism) by accounting for additional GPU requirements.

Maximum sustainable goodput R_c . To compute R_c (the maximum SLO-feasible RPS for a given trace), we perform a binary search over candidate request rates. For each candidate rate, we generate a scaled version of the input trace and run the simulator to evaluate SLO compliance. If any request violates its SLO, the rate is reduced; otherwise, it is increased. The maximum feasible rate is recorded as R_c .

To generate scaled traces, we use down-sampling (randomly dropping requests) rather than time dilation. Down-sampling preserves realistic arrival patterns and variability after load balancing, whereas time dilation artificially alters temporal correlations and leads to unrealistic per-instance workload dynamics.

Energy per request E_c . For each simulated run (e.g., at the maximum sustainable goodput R_c), we estimate energy using our power model. The simulator produces an iteration-by-iteration execution timeline. For each simulated iteration (batch) t , we predict the average GPU power P_t based on the batch features, TP degree, and frequency, and obtain the simulated iteration latency L_t from the latency model. Let N_{req} denote the number of requests completed in the simulation.

We compute the average energy per request as: $E_c = \frac{\sum_t P_t \cdot L_t}{N_{\text{req}}}$. For prefill, we additionally account for idle energy during gaps between batches using an idle-power estimate, which is important because prefill load can be bursty and instances may temporarily idle between batches.

4.3.4 Runtime Request Routing. At runtime, requests arriving at the serving cluster are routed proportionally to each instance’s maximum sustainable goodput calculated for the current configuration. For prefill, we approximate request load using prompt length and distribute requests accordingly. For decode, we treat requests as having uniform workload and route based on goodput capacity. This routing strategy ensures that the burstiness experienced by each instance aligns with the simulator’s assumptions, preserving Tier 1’s SLO guarantees.

4.4 Tier 2: Fine-Grained DVFS Control

Tier 1 provisions sufficient capacity to satisfy SLOs under peak load. However, real workloads exhibit substantial short-term variability: instantaneous request arrivals and batch compositions fluctuate even when long-term average load remains stable. Under such conditions, GPUs may operate at unnecessarily high frequencies during transient dips, resulting in avoidable energy waste.

Tier 2 addresses this inefficiency by dynamically adjusting GPU frequency at iteration-level granularity. Its objective is to exploit short-term slack while preserving the SLO guarantees ensured by Tier 1. Because prefill and decode exhibit fundamentally different dynamics (§3), we adopt a stage-aware control strategy.

Stage-Aware DVFS Strategy. As mentioned in §3, prefill and decode differ along two critical dimensions. First, prefill is compute-bound and highly sensitive to SM frequency, whereas decode is typically memory-bandwidth-bound and less responsive to frequency scaling. Second, prefill load is arrival-driven and bursty, while decode load evolves smoothly because requests persist across many iterations. These differences motivate distinct control mechanisms for the two stages: we apply model predictive control (MPC) for prefill and a simplified per-batch policy for decode.

4.4.1 Prefill: MPC-Based Frequency Control. For prefill instances, queueing delay and execution latency jointly determine TTFT. Therefore, frequency decisions affect not only the current batch but also subsequent queue evolution. As a result, myopic per-batch tuning is insufficient, and we therefore adopt an MPC approach over a finite horizon of N future batches. Specifically, at each batch boundary, the controller performs three steps:

1. **Batch projection.** Using the current waiting and running requests, along with each request’s input lengths, we simulate how they will be scheduled into the next N batches.

2. Frequency evaluation. For each candidate frequency assignment over the horizon, we use the latency model to predict batch completion times and derive projected TTFT for all active requests.
3. Feasible energy minimization. Among assignments that satisfy TTFT SLO constraints, we select the one with minimum projected energy consumption.

We approximate future uncertainty by assuming (i) no new arrivals within the horizon and (ii) ongoing requests do not finish within the horizon. We empirically find these approximations do not significantly affect the resulting system behavior. Additionally, since the controller is invoked at every batch boundary, these approximations are corrected continuously.

Efficient Frequency Search The MPC formulation requires selecting a frequency vector over N future batches. With K candidate frequencies, exhaustive search requires evaluating K^N assignments, which is infeasible in millisecond-scale control loops. To make MPC practical, we employ the greedy expansion algorithm shown in Algorithm 1.

Algorithm 1: Greedy Frequency Selection

Input: List of N available frequencies $freqs_avail$, number of future batches K

Output: Frequency assignments $freqs_opt$ for each future batch in the horizon

```

// Initialize all windows with the highest freq
1  $freqs\_opt \leftarrow [\max(freqs\_avail)]$  repeated  $K$  times;
// Iteratively expand the search space by
  including next two successive freqs
2 for  $i = 2$  to  $N - 1$  do
3    $candidates \leftarrow \emptyset$ ;
  // Mutate each usage of  $freqs\_avail[i - 1]$  to
   $freqs\_avail[i]$  and  $freqs\_avail[i + 1]$ . There are
   $3^{K'} - 1$  such mutations, where  $K'$  is the
  number of batches using  $freqs\_avail[i - 1]$ 
4   foreach valid mutation from  $freqs\_opt$  do
5      $freqs\_mutate \leftarrow mutate(freqs\_opt)$ ;
6     if  $meets\_slo(freqs\_mutate)$  then
7       Append  $freqs\_mutate$  to  $candidates$ ;
  // If exists candidate meeting SLO, pick the
  lowest average power one; otherwise, exit
  early
8   if  $candidates \neq \emptyset$  then
9      $freqs\_opt \leftarrow \arg \min_{c \in candidates} \frac{\sum_{i=1}^K latency(c, i) \cdot power(c, i)}{\sum_{i=1}^K latency(c, i)}$ 
10  else
11    break
12 return  $freqs\_opt$ 

```

The algorithm begins by assigning the maximum frequency to all N batches in the window (line 1). It then

incrementally explores lower-frequency options by progressively expanding the search space, including two additional lower frequencies at each step (line 2). We chose two instead of one, as empirically we found one would lead to the algorithm being stuck in a local minima, and two gave better results with only a small additional cost. To include the i -th and $(i + 1)$ -th candidate frequencies, starting with the current optimal frequency assignment, i.e., a vector of frequencies, one for each of the N batches, the algorithm generates a set of mutated frequency assignments, by enumerating all possible ways all occurrences of the $(i - 1)$ -th frequency in the vector can be replaced with the i -th and $(i + 1)$ -th frequency (line 4). If there are K' occurrences of the $(i - 1)$ -th frequency, this process generates $3^{K'} - 1$ candidate frequency assignments. Each candidate is then evaluated using the latency model to check compliance with TTFT SLO, and among the valid candidates, the one with the lowest power consumption is selected as the new optimal assignment (line 8). The process repeats until all candidate frequencies have been considered, and the final optimal frequency assignment is returned. If no feasible mutation exists at a given level, the search terminates early.

In practice, we select $N = 7$ frequencies from the full set supported by the GPU, and a max future horizon of $K = 8$. We empirically found that $K = 8$ can cover the request completion of all waiting requests for our chosen workloads. Compared to a brute-force approach, the algorithm reduces the worst-case complexity from K^N to $O(K \cdot 3^N)$, and after some parallelization optimizations and early exits, the average running time of the algorithm is about 4 ms.

4.4.2 Decode: Per-Batch Frequency Selection. Decode exhibits smoother load dynamics and weaker sensitivity to GPU frequency compared to prefill. Because each request persists across many iterations, the decode queue evolves gradually, and frequency decisions have limited impact on future scheduling. As a result, horizon-based MPC is unnecessary. Instead, BiScale applies a lightweight per-batch frequency selection policy.

The decode SLO is specified in terms of TPOT. However, because output length is unknown at runtime, directly predicting TPOT is difficult. We therefore use time between tokens (TBT) as a conservative proxy. Ensuring that the latency of each decode iteration satisfies the TBT constraint guarantees that the resulting TPOT also satisfies the SLO.

For each upcoming batch, the controller evaluates candidate frequencies in ascending order and selects the minimum frequency f whose predicted iteration latency satisfies the TBT constraint. If no candidate frequency satisfies the constraint, the controller falls back to the maximum frequency to preserve SLO compliance. This policy requires at most K latency model evaluations per batch, making it highly efficient for online control.

Frequency scaling also affects memory pressure indirectly. Lower frequencies increase iteration latency, prolonging request residency in the decode stage. This increases the number of concurrent requests and expands the KV cache footprint. To prevent out-of-memory (OOM) conditions, BiScale enforces a KV-cache utilization threshold. If cache usage exceeds this threshold, the controller overrides the energy-optimal frequency and temporarily selects the maximum frequency to accelerate request completion and reclaim memory.

4.5 Modeling Infrastructure

Both Tier 1 (coarse-grained provisioning) and Tier 2 (fine-grained DVFS) rely on accurate predictions of latency and energy consumption. Tier 1 uses these predictions to estimate configuration goodput and per-request energy during simulation, while Tier 2 uses them to evaluate candidate frequency assignments under SLO constraints. Since closed-form analytical models cannot capture the complex interaction between batching, tensor parallelism, frequency scaling, and workload variability, we adopt a data-driven modeling approach, by training iteration-level latency and power models offline for each (LLM model, GPU type) pair. Offline profiling is performed once and reused during runtime optimization, enabling fast decision-making without incurring measurement overhead in the serving path.

4.5.1 Latency Modeling. We build separate latency models for prefill and decode, as the two stages exhibit different performance characteristics. The latency model predicts the execution time of a single batch (or iteration) as a function of: (1) number of requests in the batch, (2) sum, mean, and standard deviation of input lengths, (3) tensor-parallel (TP) degree, and (4) GPU SM frequency. We use histogram gradient boosting trees due to their strong nonlinear modeling capacity and low inference overhead.

For training, labels are collected by instrumenting the inference engine (vLLM) to measure per-iteration execution time under different batch shapes and frequencies. For inference, for decode, per-iteration latency directly corresponds to Time Between Tokens (TBT). For prefill, batch latency contributes to TTFT together with queueing delay, which is captured by the simulator and MPC projection.

4.5.2 Power Modeling. Accurate energy estimation requires predicting power consumption at the same iteration granularity. However, power measurement presents additional challenges: GPU power APIs (e.g., NVML) report averaged power values at coarse time intervals, which do not align exactly with individual decode iterations. To address this, we collect averaged power measurements over repeated runs of similar batch configurations and use them as labels for model training.

For decode, we train a regression model using the same feature set as the latency model. We impose a monotonic

constraint along the frequency dimension, ensuring that predicted power increases with frequency, which empirically improves robustness by removing noise-induced inconsistencies in the training data.

For prefill, we observe that power behavior is well approximated by a structured interpolation. We therefore construct a three-dimensional lookup table indexed by (1) item total input token length in the batch, (2) TP degree, and (3) frequency. We then use linear interpolation between profiled points to estimate power for intermediate configurations.

We also model the GPU’s idle power, since prefill workload is bursty, and instances may remain idle between batches and consume non-negligible power.

4.6 Practical Considerations

We describe several additional experimental assumptions that affect how evaluations are conducted:

- *Future Workload Prediction.* We use a simplified approach which uses the past 5 minute load as the predicted load for the next 5 minutes. This approach assumes the workload does not change significantly over time. Other more sophisticated workload prediction algorithms may bring additional gains, but are orthogonal to our work.
- *Configuration Transition.* During the configuration transition which happens every 5 minutes, we need to tear down the old instances and spin up new instances, without interrupting the processing of the incoming requests. In this work, for simplicity, we run each 5-minute trace in isolation and report the result over the steady state. Building a complete system with minimal transition overheads can be modeled after [39].
- *Operating Margins.* GPU frequency changes using NVML typically incur tens of milliseconds delay and are not synchronized across TP GPUs. Additionally, inaccuracies in the latency prediction model result in potential SLO violations. To account for these, we add a margin of 5% for both the GPU frequency adjustment latency discussed and the target goodput discussed in §4.3. We empirically found a margin of 5% to be the sweet spot between energy savings and SLO violations.
- *Tuning for Stability* Fine-grained DVFS must remain robust to modeling inaccuracies and frequency transition delays. To prevent cascading SLO violations, we incorporate various safety margins into latency thresholds. For example, if the observed latency exceeds prediction, we immediately revert to maximum frequency. Additionally, for prefill, the controller is additionally triggered upon new request arrivals to respond quickly to bursts. Together, these mechanisms ensure that energy optimization does not compromise service reliability.

5 Implementation

We implemented BiScale on top of vLLM [18] in Python, in 1930 lines of code. The latency and power models are implemented in Python on top of Scikit-learn [35] and then converted to ONNX for faster inference. The simulator is implemented based on DistServe’s simulator, which we added additional functionality to model batch latencies at different frequencies and power modeling. We leverage NVIDIA’s NVML library [2] to perform the actual frequency scaling. To monitor the power draw, we run a separate process that asynchronously samples GPU power at fixed 10 ms intervals, using the NVML_FI_DEV_POWER_INSTANT API from NVML.

6 Evaluation

6.1 Evaluation Methodology

Workload. We construct two complementary workloads: 1) a controlled workload with steady RPS to evaluate our system under both controlled steady-state conditions, and 2) a realistic time-varying workload to exercise the system’s coarse-grained reconfiguration logic. For the controlled workload, we generate bursty arrivals by sampling inter-arrival times from a Gamma distribution with shape parameter 0.5 and a fixed average RPS. Having realistic bursty arrivals while maintaining a fixed average RPS enables controlled comparison across placement and DVFS strategies. For the realistic workload, following prior model serving works [11, 13, 15, 21, 29, 34, 39, 46], we use request timestamps from a real-world production trace – the Azure LLM inference trace [6], which exhibits multi-timescale burstiness (§2.1), and sequentially assign these timestamps to ShareGPT requests. For both workloads, we use ShareGPT [7] to obtain representative input and output length distributions.

We scale each trace to a target workload level, i.e., average RPS, to study system behavior under different load intensities. For the controlled workload, the Gamma distribution mean is directly set to the desired RPS. For the Azure-based workload, we apply time dilation to match the target average rate, which preserves the temporal structure of the original trace. This allows us to compare system behavior across a range of controlled and realistic loads.

Target workload levels are derived from measured capacity of the cluster under a serving system - we choose BiScale as we expect it outperform other baselines. First, we derive the cluster’s full capacity as the maximum RPS that can be sustained without violating TTFT or TPOT SLOs, which we obtain by performing binary search over RPS using a long request trace of 200k requests. Specifically, at each binary search iteration, we run the coarse-grained placement algorithm to derive a cluster configuration, and use it to serve the requests with the fine-grained DVFS system enabled, and check end-to-end SLO compliance. Second, after determining the cluster’s capacity above, we evaluate each baseline at 67% and 85% of this value. Operating below peak capacity

is consistent with production practice at Meta [3] and Microsoft [33], where clusters are typically run with headroom to absorb burstiness and transient load spikes.

GPU & LLM Model. All experiments are conducted on two nodes on Nebius cloud [1], each equipped with 8 NVIDIA H100 GPUs connected via InfiniBand, for a total of 16 GPUs. As the serving model, we use Llama 3.3 70B [26], a widely used mid-scale Llama model.

Metrics and SLOs. Following standard practice in LLM serving, we report TTFT and TPOT as latency metrics. For TTFT, we report the 99th percentile (P99) across requests. For TPOT, we first compute the mean TPOT per request and then report the P99 across requests. We set the TTFT SLO to 600 ms and the TPOT SLO to 100 ms. The TTFT target is derived from the SLO used in DistServe for a similarly sized model, adjusted to reflect performance differences between A100 and H100 GPUs.

For power draw, we report the average GPU power (in watts) over the duration of each session, summed across all GPUs. For energy efficiency, we report normalized energy in joules per token. Specifically, prefill energy is normalized by the number of processed requests (each request produces one first token), and decode energy is normalized by the total number of generated output tokens.

To ensure measurement consistency, we exclude the ramp-up and ramp-down phases of each session and compute all metrics using only the steady-state portion. We define the ramp-up region as the period between session start and 30 seconds after session start, which we empirically find sufficient for the system to reach steady state. We define the ramp-down region as the period between the end of request issuance and session end (i.e., when all in-flight requests are drained).

Baselines. We compare BiScale against the following baselines:

- **DistServe:** This baseline follows the configuration strategy described in [46], which generates a placement that satisfies TTFT and TPOT SLOs while maximizing throughput per GPU. All GPUs operate at the maximum supported frequency.
- **PlaceOnly:** This baseline applies only the Tier 1 coarse-grained provisioning described in §4.3, selecting a placement that minimizes energy consumption subject to SLO constraints. Unlike DistServe, it allows GPUs to operate at fixed frequencies that may be lower than the maximum. However, it does not perform fine-grained DVFS during execution.

6.2 End-to-end Results

We conduct end-to-end evaluations of BiScale using two workloads described in §6.1: a controlled workload and a realistic time-varying workload.

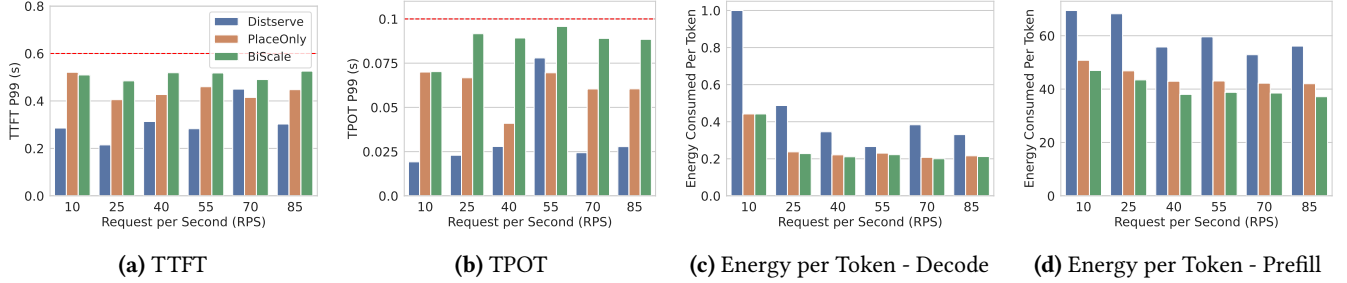


Figure 5. Results for various controlled workloads with constant average RPS

Table 1. Cluster placement for 5–10 minute period. Note that BiScale uses the same cluster configuration as PlaceOnly, so it is not listed separately.

Load	System	Phase	TP	Frequency (GHz)	Load balancing weights (%)
67%	DistServe	Prefill	$3 \times TP2$	1.83, 1.83, 1.83	33.3, 33.3, 33.3
		Decode	$2 \times TP4$	1.83, 1.83	50.0, 50.0
	PlaceOnly	Prefill	$4 \times TP2$	1.08, 1.08, 1.35, 1.23	22.2, 22.2, 29.3, 26.3
		Decode	$2 \times TP4$	1.08, 1.35	46.0, 54.0
85%	DistServe	Prefill	$4 \times TP2$	1.83, 1.83, 1.83, 1.83	25.0, 25.0, 25.0, 25.0
		Decode	$2 \times TP4$	1.83, 1.83	50.0, 50.0
	PlaceOnly	Prefill	$4 \times TP2$	1.83, 1.83, 1.32, 1.32	26.8, 26.8, 23.2, 23.2
		Decode	$2 \times TP4$	1.56, 1.47	52.0, 48.0

6.2.1 Controlled Workload. We generate the controlled workload following §6.1, varying the target RPS from 10 to 85 in increments of 15 RPS. The resulting cluster configurations are listed in Appendix Table 2, and the TTFT, TPOT, and energy consumption for both prefill and decode are reported in Figure 5.

BiScale satisfies TTFT and TPOT SLOs across all evaluated RPS values. As shown in the two left plots of Figure 5, P99 TTFT and TPOT remain below their respective SLO thresholds (indicated by red lines). Compared to the two baselines, BiScale typically exhibits higher P99 latency values while still meeting SLOs. This reflects its DVFS strategy, which elongates iterations as much as possible without violating latency constraints in order to reduce energy consumption.

In terms of energy efficiency, DistServe consistently consumes the highest energy per token. For decode, PlaceOnly and BiScale achieve comparable energy consumption. For prefill, BiScale achieves the lowest energy consumption, followed by PlaceOnly and then DistServe. Specifically, PlaceOnly reduces prefill energy by 20–31% relative to DistServe, while BiScale achieves a larger reduction of 27–36%. We analyze the underlying causes of these differences in §6.3.

6.2.2 Production Workload. We next evaluate the systems using the realistic workload described in §6.1, scaled to 67% and 85% of full capacity as described in §6.1. This workload captures realistic, time-varying request arrivals and exercises the system’s coarse-grained reconfiguration mechanism. The scaled traces are shown in Figure 6, where

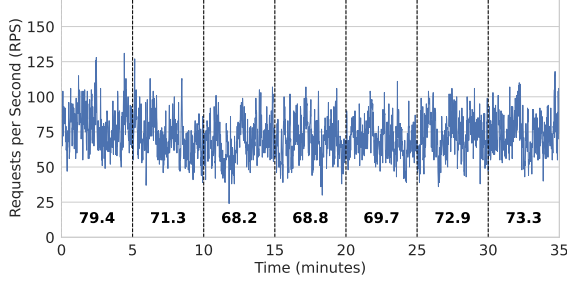
vertical lines denote 5-minute intervals at which Tier 1 re-configuration occurs. Since each configuration is generated based on the load observed in the preceding 5-minute window, we exclude the initial 0–5 minute interval and report results starting from minute 5. The configuration for the first evaluated window (minutes 5–10) is shown in Table 1, while subsequent configurations are listed in Appendix Table 2.

The results are shown in Figure 7, where the top and bottom rows correspond to 67% and 85% workload, respectively. We observe that all systems satisfy TTFT and TPOT SLOs under both load levels, with one minor exception: under the 85% workload, PlaceOnly exceeds the TPOT SLO by 1.5 ms during the 20–25 minute window. Consistent with the controlled workload setting, BiScale operates closer to the SLO boundaries, exhibiting higher P99 TTFT and TPOT values while remaining within latency constraints.

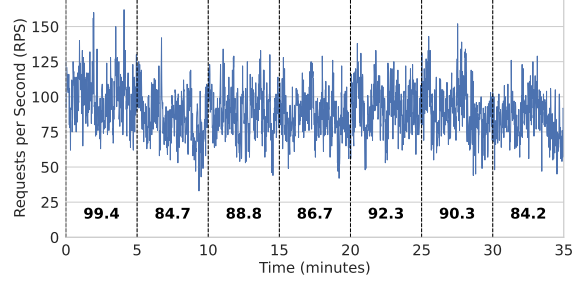
Energy trends are consistent between the 67% and 85% workload settings. For decode, DistServe consumes the most energy, while PlaceOnly and BiScale achieve comparable reductions. Under 67% load, PlaceOnly reduces decode energy by 37–45% relative to DistServe, while BiScale achieves a 44–48% reduction. In contrast, for prefill, BiScale consistently achieves the lowest energy consumption, followed by PlaceOnly and then DistServe. Under 67% load, PlaceOnly reduces prefill energy by 16–29% relative to DistServe, while BiScale achieves larger reductions of 28–39%. Similar trends are observed at 85% load.

6.3 Energy Saving Analysis

6.3.1 Impact of Workload Prediction. We next analyze where the additional savings of BiScale (over PlaceOnly) come from. Figure 5 and Figure 7 show that most savings are obtained by selecting an energy-efficient static configuration for each workload window (i.e., PlaceOnly), which reduces the energy of DistServe by 11% to 29% (20% on average) for prefill, and 16% to 45% (33% on average) for decode. DVFS provides a smaller incremental gain: ranging from -4% to 20% (6% on average) for decode, and 9% to 29% (15% on average) for prefill. Note that the -4% energy gain is from a 5-minute window where PlaceOnly violated the SLO. Hence, most of the incremental savings from DVFS come from prefill, which contributes about 2.5× more than decode.

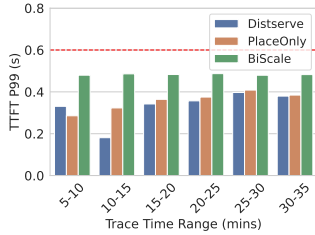


(a) Workload scaled to 67% of the full capacity.

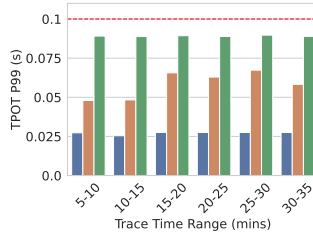


(b) Workload scaled to 85% of the full capacity.

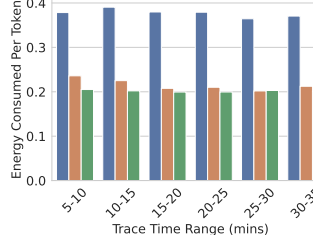
Figure 6. The production workload in terms of RPS, divided into 5-minute chunks.



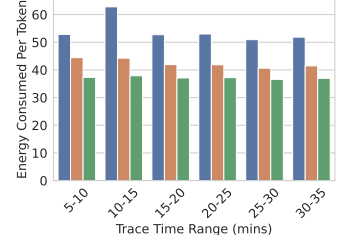
(a) TTFT



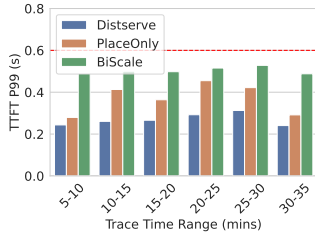
(b) TPOT



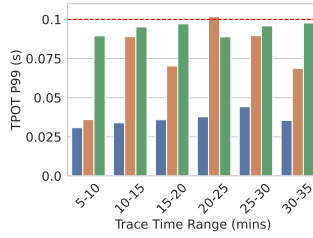
(c) Energy per token - Decode



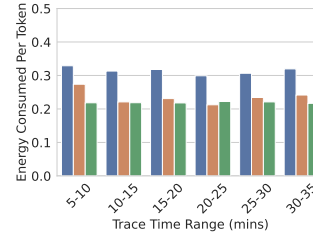
(d) Energy per token - Prefill



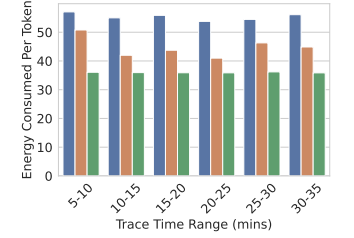
(e) TTFT



(f) TPOT



(g) Energy per token - Decode



(h) Energy per token - Prefill

Figure 7. Results for 30 minute long production traces at 67% (1st row) and 85% (2nd row) capacity of the system

Comparing controlled and production traces clarifies the role of workload prediction in placement. Under controlled traces, decode savings over PlaceOnly are only 2.8% (vs. 8.6% for prefill), whereas under production traces they increase to 6.3% for decode (and 15.5% for prefill). In controlled traces, PlaceOnly is derived from the known trace (for the current window) and is therefore close to optimal, leaving little headroom for DVFS. In production traces, we predict each next 5-minute workload from the previous 5-minute window; this introduces both over-prediction and under-prediction (Figure 6). DVFS is therefore useful as a correction mechanism that compensates for prediction error online, which explains the larger gains under realistic workloads.

To further localize these effects, we next examine individual 5-minute windows and compare system dynamics for decode and prefill. This window-level analysis reveals where the savings come from and why DVFS yields smaller gains for decode than for prefill.

6.3.2 Decode. We first analyze decode at the granularity of individual 5-minute windows to explain why DVFS has lower

energy-saving potential than in prefill. We focus on two representative windows from the 85% workload: (i) an over-configured window (5–10 minutes), where BiScale achieves clear savings over PlaceOnly, and (ii) an under-configured window (20–25 minutes), where BiScale consumes more energy than PlaceOnly.

Figure 8 and Figure 9 show internal dynamics for these two windows. Each figure includes six time-series plots: input RPS, per-instance frequency (two instances), per-instance decode power, total GPU power, batch size, and batch latency. As discussed in §6.1, we treat the first 30 seconds as ramp-up and exclude it from energy and latency metrics.

In the over-configured window (Figure 8), PlaceOnly selects higher fixed frequencies for both decode instances than BiScale. This over-provisioning comes from load over-prediction in the previous window, so PlaceOnly runs at unnecessarily high fixed frequencies. In contrast, BiScale applies DVFS online to lower frequency when slack exists, mitigating the energy penalty of over-provisioning. As a

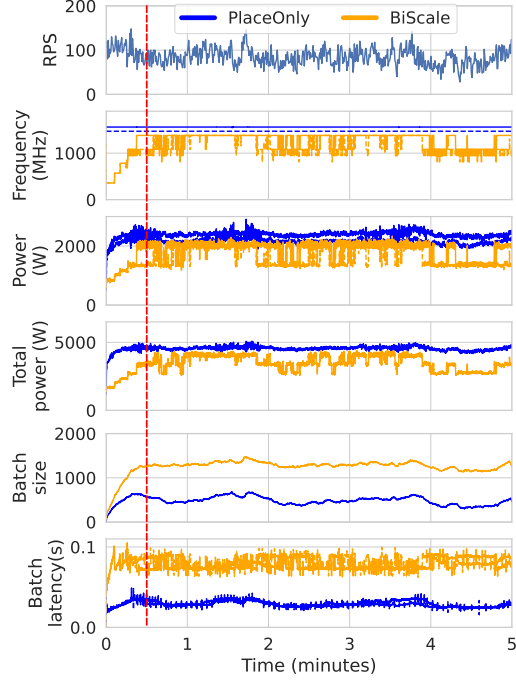


Figure 8. Decode over-configuration case (5-10 minutes 85% capacity workload).

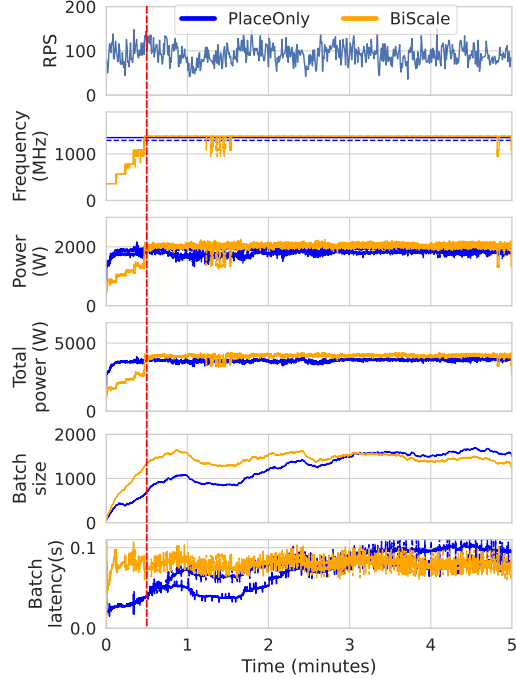


Figure 9. Decode under-configuration case (20-25 minutes 85% capacity workload).

result, BiScale draws less power over the window and consumes less total energy.

In the under-configured window (Figure 9), both PlaceOnly and BiScale configuration is determined by the lower RPS in the previous 5-minute window and is

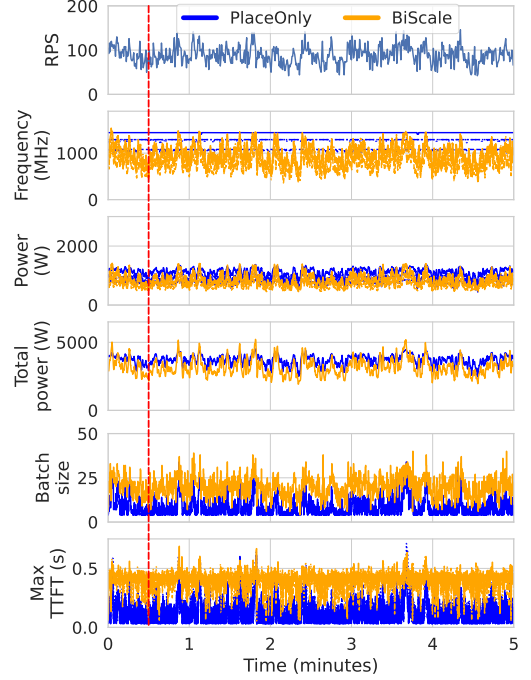


Figure 10. Prefill phase timelines in the 10-15 minute window, under 85% capacity workload.

therefore too under-provisioned for the current demand. Since PlaceOnly operates at the fixed pre-determined frequency, it cannot raise the frequency to account for under-provisioning, and eventually violates the TPOT SLO during minutes 4-5. In contrast, with the same under-provisioned cluster configuration, BiScale dynamically adjust the frequency based on transient workload change, and operates at a slightly higher frequency than PlaceOnly, to meet the SLO.

In summary, decode savings depend on whether the static configuration over- or under-provisions the next 5-minute window. When PlaceOnly over-provisions, BiScale lowers frequency and energy; when PlaceOnly under-provisions, BiScale may raise frequency to protect SLOs, reducing gains.

6.3.3 Prefill. For prefill, as was shown in §6.2.2, BiScale achieves the lowest energy consumption in all 5-minute windows. We therefore present one representative window (10–15 minutes) in Figure 10, using the same timeline views as in the decode analysis.

Immediately, we observe that compared to decode, prefill batch size tracks input workload much more closely; bursty RPS directly translates to bursty batches. This variation in compute demand drives frequency and thus power variations in BiScale, while PlaceOnly keeps a fixed frequency. During low-load periods, BiScale can reduce frequency aggressively to save energy, while during bursts or backlog growth it can temporarily raise frequency to protect SLOs. As a result, BiScale exhibits a more adaptive power profile and lower overall energy. The maximum TTFT under BiScale

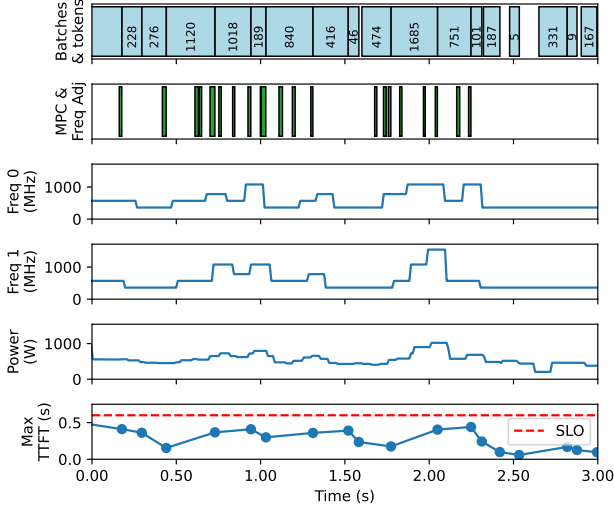


Figure 11. Microscopic behavior of the prefill instance.

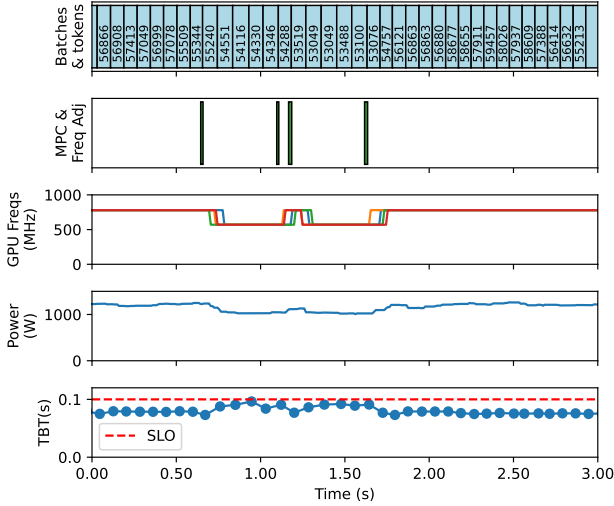


Figure 12. Microscopic behavior of the decode instance. In the top figure, each request count as N tokens, where N is the running request length.

also stays closer to the SLO target, indicating that BiScale reduces energy while still meeting latency constraints.

6.4 Microscopic View of DVFS

After analyzing 5-minute timelines, we next zoom in to sub-second behavior to show how BiScale operates online for prefill and decode in Figure 11 and Figure 12, respectively. In both figures, from top to bottom, we show: (1) individual batches as rectangles annotated with batch token count; (2) MPC events as rectangles, where each rectangle marks one MPC/DVFS invocation (we only plot invocations that changed frequency to avoid clutter); (3-4) frequency of each TP rank, where prefill uses a TP 2 instance, and decode uses TP 4; (5) total system power draw summed across prefill

GPUs or decode GPUs for each case; and (6) per-batch latency metric, i.e., TTFT for prefill and TBT for decode, with one dot per batch.

Prefill. When the load is low, e.g., before 0.5 s during batches of shape 228 and 276, BiScale applies the minimum frequency and power drops accordingly. As request pressure increases, queueing grows and BiScale raises frequency, e.g., from 360 MHz to 570 and then 780 MHz during the batch of shape 474, all of which were the lowest frequency that could satisfy the SLO; the maximum TTFT remains below the 600 ms target. Most frequency updates occur at batch boundaries or when enough new requests arrive during an executing batch. We also observe that frequency sometimes change within batches (§4.6) due to new request arrivals, e.g., as seen in the batch of shape 474 at around 1.7 seconds. Additionally, we observe an under-prediction event around 2 seconds (the batch of size 1685): the executing batch runs longer than predicted, BiScale thus immediately switches to maximum frequency to mitigate the effect of under-prediction, and then lowers frequency again at the next batch boundary.

One important detail is that GPUs in an instance do not always follow identical frequency trajectories, and observed frequency changes can lag MPC invocation times. This is expected for two reasons: frequency application latency (discussed in §4.6) and boost-frequency behavior (frequencies above 1095 MHz are boost states and are sustained only when thermal and power headroom permit). Thermal and power characteristics are dynamic and can vary across GPUs even within one node, leading to small per-rank differences.

Decode. Figure 12 shows that frequency changes are much less frequent than in prefill because decode batch size evolves more gradually over time. As requests arrive and finish, BiScale alternates between higher and lower frequencies to meet the SLO while minimizing energy. The power trace follows these adjustments closely: lowering frequency reduces power, and raising frequency restores it. In decode, opportunities to run at lower frequency are short and intermittent, so the incremental DVFS energy benefit is correspondingly modest.

6.5 Latency/Power Model Accuracy

Accurate latency and power models are essential because DVFS decisions and placement quality both depend directly on these predictions. We use iteration-level data collected in the constant-frequency experiments (DistServe and PlaceOnly in §6.2.1) to evaluate model accuracy. Figure 13a and Figure 13b compare predicted and measured iteration latency. The latency model achieves mean absolute percentage error (MAPE) of 2.9% for prefill instances and 2.7% for decode instances.

To evaluate the power model, we compare predicted and measured energy over consecutive 10-second windows, as shown in Figure 13c and Figure 13d for prefill and decode,

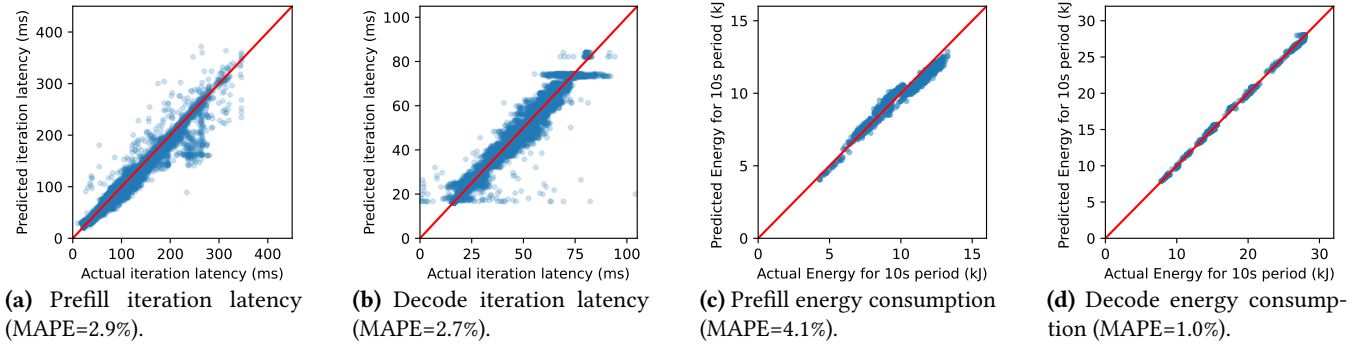


Figure 13. Accuracy of latency and power models against measured values.

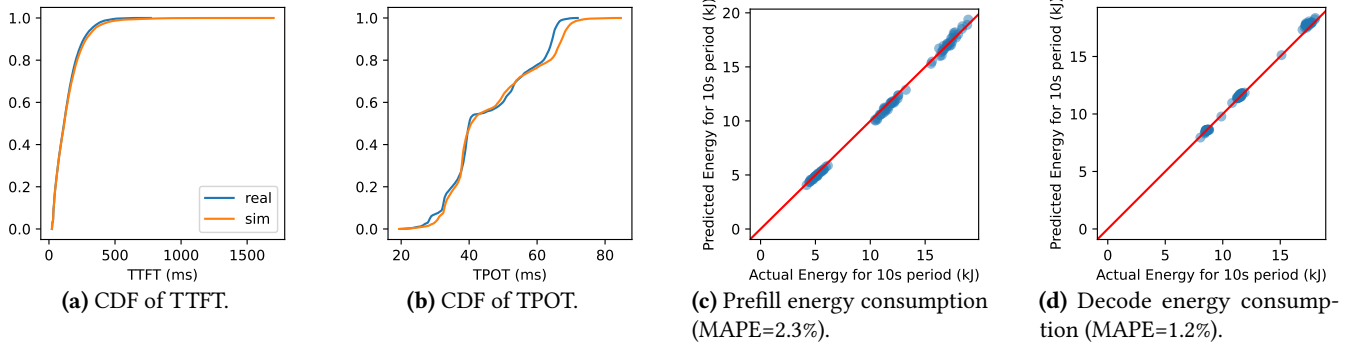


Figure 14. Accuracy of the Tier-1 simulator against real runs, shown by TTFT CDF, TPOT CDF, and energy comparisons.

respectively. We use this granularity because power is sampled every 100 ms and measurements are often not aligned to iteration boundaries. Overall, the power model achieves MAPE of 4.1% for prefill instances and 1.0% for decode instances. These errors are small enough for both online DVFS control and placement generation.

6.6 Simulation Accuracy

Accurate simulation by the simulator is essential because the Tier-1 placement algorithm uses it to evaluate candidate placements before deployment. We evaluate simulation accuracy using the 10 RPS, 25 RPS, and 40 RPS traces from §6.2.1. For each trace, the simulator uses the same frequency set as the placement produced by the Tier-1 placement algorithm. We then compare simulator outputs against latency and power measurements collected from the real system under the same traces. Figure 14a and Figure 14b compare the CDFs of simulated and measured TTFT and TPOT. TTFT CDFs are closely aligned. We see that TPOT CDFs are also broadly aligned, with the largest deviation in the top 10% tail where simulation is slightly higher than measurement, which makes placements mildly conservative while still preserving SLO validity.

We next compare simulated and measured cumulative energy for each consecutive 10-second window. Figure 14c and Figure 14d compare prefill and decode energy between simulation and the real engine. The simulated prefill energy

achieves a MAPE of 2.3%, and the simulated decode energy achieves a MAPE of 1.2%. These results indicate that simulator fidelity is sufficient for Tier-1 placement decisions.

7 Related Work

Energy-efficient LLM serving. Several recent works have explored reducing the energy usage of LLM inference through placement and parallelism optimizations or GPU frequency tuning. DynamoLLM [39] divides GPUs into pools that serve requests with similar input and output lengths and optimizes parallelism configurations within each pool to reduce energy consumption. However, when computing placements, it assumes that all GPUs within a pool operate at the same frequency and parallelism settings, rather than modeling these as decision variables. throttLL’eM [14] applies predictive GPU throttling to reduce energy while meeting latency SLOs, but assumes fixed serving placement and parallelism settings. Neither system considers prefill–decode disaggregation, where prefill and decode exhibit distinct burstiness patterns, bottlenecks, and SLO sensitivities.

Energy-efficient LLM training. Prior work on energy efficiency in LLM training, such as Zeus [42] and Perseus [8], leverages DVFS to reduce energy by exploiting iteration-level slack in long-running and predictable workloads. These training-oriented systems do not directly transfer to online

serving, which must handle bursty workloads while satisfying strict latency SLOs.

General LLM serving systems. A large body of work improves LLM serving performance through batching, scheduling, parallelism, and prefill–decode disaggregation [4, 18, 20, 22, 24, 34, 37, 41, 43, 46]. These systems primarily optimize throughput or latency. Our approach is orthogonal and complementary: it targets energy reduction through DVFS while preserving serving SLOs.

8 Conclusion

In this paper, we presented BiScale, a two-tier framework for energy-efficient disaggregated LLM serving that jointly optimizes placement and DVFS across prefill and decode while preserving strict TTFT and TPOT SLOs. BiScale combines model-driven, phase-aware placement at coarse timescales with fine-grained, stage-specific DVFS control to exploit workload slack and correct prediction errors safely.

Evaluation on a multi-GPU cluster using production-style workloads shows that BiScale substantially reduces energy consumption compared to state-of-the-art disaggregated serving systems while consistently meeting latency SLOs. These results demonstrate that coordinated, phase-aware control across placement and DVFS is essential for improving the energy efficiency of disaggregated LLM serving.

References

- [1] [n. d.]. Nebius AI Cloud Platform. <https://nebius.com/>.
- [2] 2025. NVIDIA Management Library (NVML). <https://developer.nvidia.com/management-library-nvml>.
- [3] 2025. Taming the tail utilization of ads inference at Meta scale. https://engineering.fb.com/2024/07/10/production-engineering/tail-utilization-ads-inference-meta/?utm_source=chatgpt.com.
- [4] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming {Throughput-Latency} tradeoff in {LLM} inference with {Sarathi-Serve}. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 117–134.
- [5] Anthropic. 2025. Claude Models Overview. <https://docs.anthropic.com/en/docs/about-claude/models/overview>.
- [6] Azure. 2025. Azure Public Dataset. <https://github.com/Azure/AzurePublicDataset>.
- [7] Lin Chen, Jinsong Li, Xiaoyi Dong, Pan Zhang, Conghui He, Jiaqi Wang, Feng Zhao, and Dahua Lin. 2024. Sharegpt4v: Improving large multi-modal models with better captions. In *European Conference on Computer Vision*. Springer, 370–387.
- [8] Jae-Won Chung, Yile Gu, Insu Jang, Luoxi Meng, Nikhil Bansal, and Mosharaf Chowdhury. 2024. Reducing energy bloat in large model training. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles*. 144–159.
- [9] Daniel Crankshaw, Gur-Eyal Sela, Xiangxi Mo, Corey Zumar, Ion Stoica, Joseph Gonzalez, and Alexey Tumanov. 2020. InferLine: latency-aware provisioning and scaling for prediction serving pipelines. In *Proc. of ACM SoCC*. 477–491.
- [10] Daniel Crankshaw, Xin Wang, Guanyu Zhou, Michael J Franklin, Joseph E Gonzalez, and Ion Stoica. 2020. InferLine: ML Inference Pipeline Provisioning and Management for Tight Latency SLOs. In *14th USENIX Symposium on Operating Systems Design and Implementation*. 283–300.
- [11] Yao Fu, Leyang Xue, Yeqi Huang, Andrei-Octavian Brabete, Dmitrii Ustiugov, Yuvraj Patel, and Luo Mai. 2024. {ServerlessLLM}:{Low-Latency} serverless inference for large language models. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 135–153.
- [12] Mark W Garrett and Walter Willinger. 1994. Analysis, modeling and generation of self-similar VBR video traffic. *ACM SIGCOMM computer communication review* 24, 4 (1994), 269–280.
- [13] Ruihao Gong, Shihao Bai, Siyu Wu, Yunqian Fan, Zaijun Wang, Xiuhong Li, Hailong Yang, and Xianglong Liu. 2025. Past-Future Scheduler for LLM Serving under SLA Guarantees. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 798–813.
- [14] Andreas Kosmas Kakolyris, Dimosthenis Masouros, Petros Vavaroutos, Sotirios Xydis, and Dimitrios Soudris. 2025. throttLL’eM: Predictive GPU Throttling for Energy Efficient LLM Inference Serving. In *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 1363–1378. doi:10.1109/HPCA61900.2025.00103
- [15] Andreas Kosmas Kakolyris, Dimosthenis Masouros, Sotirios Xydis, and Dimitrios Soudris. 2024. SLO-Aware GPU DVFS for Energy-Efficient LLM Inference Serving. *IEEE Computer Architecture Letters* 23, 2 (2024), 150–153. doi:10.1109/LCA.2024.3406038
- [16] Joshua Karp, Jinwoo Kim, Grace Liu, et al. 2022. Autoscaling Large Language Models with Demand Forecasting and Inference-Aware Resource Management. *arXiv preprint arXiv:2210.06423* (2022).
- [17] Kimi Team. 2025. Kimi K2 Technical Report. *arXiv preprint arXiv:2507.20534* (2025).
- [18] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*. 611–626.
- [19] Will E Leland, Murad S Taqqu, Walter Willinger, and Daniel V Wilson. 2002. On the self-similar nature of Ethernet traffic (extended version). *IEEE/ACM Transactions on networking* 2, 1 (2002), 1–15.
- [20] Jiamin Li, Yimin Jiang, Yibo Zhu, Cong Wang, and Hong Xu. 2023. Accelerating distributed {MoE} training and inference with lina. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*. 945–959.
- [21] Zhuohan Li, Lianmin Zheng, Yinmin Zhong, Vincent Liu, Ying Sheng, Xin Jin, Yanping Huang, Zhifeng Chen, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. AlpaServe: Statistical Multiplexing with Model Parallelism for Deep Learning Serving. In *Proc. of USENIX OSDI*. USENIX Association, Boston, MA, 663–679. <https://www.usenix.org/conference/osdi23/presentation/li-zhouhan>
- [22] Chaofan Lin, Zhenhua Han, Chengruidong Zhang, Yuqing Yang, Fan Yang, Chen Chen, and Lili Qiu. 2024. Parrot: Efficient serving of {LLM-based} applications with semantic variable. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 929–945.
- [23] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. 2024. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437* (2024).
- [24] Yuhao Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang, Kuntai Du, Jiayi Yao, Shan Lu, Ganesh Ananthanarayanan, et al. 2024. CacheGen: Kv cache compression and streaming for fast large language model serving. In *Proceedings of the ACM SIGCOMM 2024 Conference*. 38–56.
- [25] Tania Llorido-Botran, Jose Miguel-Alonso, and Jose A Lozano. 2014. A review of auto-scaling techniques for elastic applications in cloud environments. *Journal of grid computing* 12, 4 (2014), 559–592.
- [26] Meta. 2024. Meta Llama 3. <https://llama.meta.com/llama3>.
- [27] Microsoft Research. 2025. The growing energy footprint of AI inference. <https://www.microsoft.com/en-us/research/publication/energy>

- use-of-ai-inference-efficiency-pathways-and-test-time-compute/.
- [28] Arvind Narayanan, Siddharth Bandyopadhyay, et al. 2023. The Cost of Serving Large Language Models. *arXiv preprint arXiv:2309.00696* (2023).
 - [29] Chengyi Nie, Rodrigo Fonseca, and Zhenhua Liu. 2024. Aladdin: Joint Placement and Scaling for SLO-Aware LLM Serving. *arXiv preprint arXiv:2405.06856* (2024).
 - [30] Chenxu Niu, Wei Zhang, Yongjian Zhao, and Yong Chen. 2025. Energy Efficient or Exhaustive? Benchmarking Power Consumption of LLM Inference Engines. 5, 2 (Aug. 2025), 56–62. doi:10.1145/3757892.3757900
 - [31] NVIDIA. 2025. NVIDIA Dynamo Disaggregated Serving Delivers 30x More Inference on Reasoning AI Models. <https://developer.nvidia.com/blog/nvidia-dynamo-disaggregated-serving-delivers-30x-more-inference-on-reasoning-ai-models/>.
 - [32] OpenAI. 2025. GPT-5. <https://openai.com/gpt-5>.
 - [33] Pratyush Patel, Esha Choukse, Chaojie Zhang, Íñigo Goiri, Brijesh Warriar, Nithish Mahalingam, and Ricardo Bianchini. 2024. Characterizing power management opportunities for llms in the cloud. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. 207–222.
 - [34] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient generative llm inference using phase splitting. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 118–132.
 - [35] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. 2011. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research* 12 (2011), 2825–2830.
 - [36] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. 2022. DeepSpeed-MoE: Advancing Mixture-of-Experts Inference and Training to Power Next-Generation AI Scale. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*.
 - [37] Yixin Song, Zeyu Mi, Haotong Xie, and Haibo Chen. 2024. Powerinfer: Fast large language model serving with a consumer-grade gpu. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles*. 590–606.
 - [38] Jovan Stojkovic, Esha Choukse, Chaojie Zhang, Inigo Goiri, and Josep Torrellas. 2024. Towards greener llms: Bringing energy-efficiency to the forefront of llm inference. *arXiv preprint arXiv:2403.20306* (2024).
 - [39] Jovan Stojkovic, Chaojie Zhang, Íñigo Goiri, Josep Torrellas, and Esha Choukse. 2025. Dynamollm: Designing llm inference clusters for performance and energy efficiency. In *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 1348–1362.
 - [40] vLLM Project. 2025. Disaggregated Prefill V1. https://docs.vllm.ai/en/latest/features/disagg_prefill.html.
 - [41] Bingyang Wu, Shengyu Liu, Yinmin Zhong, Peng Sun, Xuanzhe Liu, and Xin Jin. 2024. Loongserve: Efficiently serving long-context large language models with elastic sequence parallelism. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles*. 640–654.
 - [42] Jie You, Jae-Won Chung, and Mosharaf Chowdhury. 2023. Zeus: Understanding and optimizing {GPU} energy consumption of {DNN} training. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. 119–139.
 - [43] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A distributed serving system for {Transformer-Based} generative models. In *Proc. of USENIX OSDI*. 521–538.
 - [44] Yifan Yuan, Yuzhe Liu, Soumya Ghosh, et al. 2023. Sage: Predictive Autoscaling for ML Inference Workloads. In *Proceedings of the 17th*

USENIX Symposium on Operating Systems Design and Implementation.

- [45] Yifan Zhao, Shijie Lin, and Zhihao Jia. 2023. AutoScale-LLM: Cost-Efficient Large Language Model Inference with Predictive Autoscaling. arXiv:2310.06474 [cs.DC] Available at <https://arxiv.org/abs/2310.06474>.
- [46] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. {DistServe}: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. In *Proc. of USENIX OSDI* (2024). 193–210.

Appendix

A Placement Configurations

Table 2 lists the full Tier 1 placement plans used in the time-varying production-trace experiments reported in §6.2. It reports, for each load level (67% and 85%) and each 5-minute window from 5–10 to 30–35 minutes, the selected prefill/decode configuration for DistServe and PlaceOnly. Each table entry includes the number and TP degree of instances, the per-instance GPU frequencies (GHz), and the router load-balancing weights used during that window.

Table 2. TP degree, frequency, and load balancing weights for prefill and decode for each 5 minute period. Note that BiScale uses the same cluster configuration as Min Energy, so it is not listed separately.

Load		5–10 mins	10–15 mins	15–20 mins	20–25 mins	25–30 mins	30–35 mins
67%	Distserve Prefill	3×TP2	4×TP2	3×TP2	3×TP2	3×TP2	3×TP2
		1.83,1.83,1.83	1.83,1.83,1.83,1.83	1.83,1.83,1.83	1.83,1.83,1.83	1.83,1.83,1.83	1.83,1.83,1.83
	Min Energy Prefill	33.3,33.3,33.3	25.0,25.0,25.0,25.0	33.3,33.3,33.3	33.3,33.3,33.3	33.3,33.3,33.3	33.3,33.3,33.3
		4×TP2	4×TP2	4×TP2	4×TP2	4×TP2	4×TP2
	Min Energy Prefill	1.08,1.08,1.23,1.35	1.08,1.08,1.08,1.29	1.08,1.08,1.11,1.11	1.08,1.08,1.08,1.08	1.08,1.08,1.08,1.11	1.17,1.11,1.11,1.05
		22.1,22.1,29.7,26.1	23.7,23.7,23.7,28.8	24.1,24.1,25.9,25.9	25.0,25.0,25.0,25.0	24.8,24.8,24.8,25.6	27.0,24.8,24.8,23.5
	Distserve Decode	2×TP4	2×TP4	2×TP4	2×TP4	2×TP4	2×TP4
		1.83,1.83	1.83,1.83	1.83,1.83	1.83,1.83	1.83,1.83	1.83,1.83
	Min Energy Decode	25.0,25.0	25.0,25.0	25.0,25.0	25.0,25.0	25.0,25.0	25.0,25.0
		2×TP4	2×TP4	2×TP4	2×TP4	2×TP4	2×TP4
	Min Energy Decode	1.08,1.35	1.17,1.05	1.05,1.05	1.11,1.05	1.08,1.08	1.08,1.17
		41.5,58.5	52.1,47.9	50.0,50.0	51.8,48.2	50.0,50.0	48.4,51.6
85%	Distserve Prefill	4×TP2	4×TP2	4×TP2	4×TP2	4×TP2	4×TP2
		1.83,1.83,1.83,1.83	1.83,1.83,1.83,1.83	1.83,1.83,1.83,1.83	1.83,1.83,1.83,1.83	1.83,1.83,1.83,1.83	1.83,1.83,1.83,1.83
	Min Energy Prefill	25.0,25.0,25.0,25.0	25.0,25.0,25.0,25.0	25.0,25.0,25.0,25.0	25.0,25.0,25.0,25.0	25.0,25.0,25.0,25.0	25.0,25.0,25.0,25.0
		4×TP2	4×TP2	4×TP2	4×TP2	4×TP2	4×TP2
	Min Energy Prefill	1.83,1.83,1.32,1.32	1.08,1.29,1.29,1.44	1.11,1.38,1.38,1.38	1.08,1.26,1.38,1.38	1.08,1.44,1.5,1.5	1.44,1.32,1.32,1.32
		26.8,26.8,23.2,23.2	20.2,25.7,25.7,28.5	20.7,26.4,26.4,26.4	20.7,24.8,27.3,27.3	20.2,26.5,26.6,26.6	26.3,24.6,24.6,24.6
	Distserve Decode	2×TP4	2×TP4	2×TP4	2×TP4	2×TP4	2×TP4
		1.83,1.83	1.83,1.83	1.83,1.83	1.83,1.83	1.83,1.83	1.83,1.83
	Min Energy Decode	25.0,25.0	25.0,25.0	25.0,25.0	25.0,25.0	25.0,25.0	25.0,25.0
		2×TP4	2×TP4	2×TP4	2×TP4	2×TP4	2×TP4
	Min Energy Decode	1.56,1.47	1.11,1.47	1.08,1.56	1.35,1.29	1.23,1.56	1.2,1.56
		52.0,48.0	43.6,56.4	41.6,58.4	50.7,49.3	43.0,57.0	43.8,56.2